

Proyecto Final

Plataforma de Gestión Clínica “ClinicFlow”

Profesor: Jorge Zambrano Ibujés

Auxiliar: Christian Díaz Guerra

Ayudante: Pablo Vergara Llantén

1. Contexto y motivación

Actualmente, gran parte de la gestión administrativa y operacional de clínicas y centros médicos pequeños se realiza mediante procesos manuales, llamados telefónicos, planillas dispersas y múltiples sistemas desconectados. Esto genera problemas frecuentes tales como: pérdida de horas médicas, cancelaciones tardías, sobrecarga de recepcionistas, baja capacidad de respuesta, errores de coordinación, agendas inconsistentes y dificultades para administrar listas de espera.

En paralelo, los asistentes conversacionales basados en inteligencia artificial han comenzado a utilizarse como mecanismo de automatización para atención de pacientes y coordinación clínica (por ejemplo, **Cero.ai**). Sin embargo, la complejidad de estos sistemas no radica únicamente en la integración con modelos de lenguaje, sino también en el correcto modelado del dominio, las reglas de negocio, los workflows clínicos y la arquitectura del sistema.

El objetivo de este proyecto es desarrollar una plataforma web funcional de gestión clínica conversacional que permita automatizar parte importante de los procesos administrativos de una clínica mediante asistentes inteligentes integrados con Telegram, aplicando conceptos avanzados de Programación Orientada a Objetos, testing, arquitectura de software y despliegue.

2. Objetivos

Los objetivos de este proyecto son:

- Diseñar un sistema complejo utilizando Programación Orientada a Objetos.
- Aplicar correctamente: herencia, polimorfismo, abstracción, encapsulamiento y composición.
- Diseñar un sistema basado en eventos y automatizaciones.
- Modelar un dominio con múltiples entidades y reglas de negocio.
- Diseñar una arquitectura mantenible y extensible.
- Implementar pruebas automatizadas.
- Separar responsabilidades entre dominio, lógica de negocio, persistencia y presentación.

- Desplegar una aplicación web funcional.
- Utilizar herramientas de IA de manera efectiva para apoyar el desarrollo del sistema.

3. Descripción del proyecto

Se deberá implementar una plataforma web para administrar procesos clínicos mediante interacción conversacional automatizada.

El sistema deberá permitir:

- Gestionar usuarios y permisos.
- Administrar clínicas, especialidades y agendas médicas.
- Gestionar citas médicas.
- Confirmar, cancelar y reagendar horas.
- Manejar listas de espera.
- Gestionar bloqueos y suspensiones de agenda.
- Derivar pacientes entre especialidades.
- Registrar conversaciones realizadas por asistentes IA.
- Validar restricciones y reglas de negocio.
- Visualizar información operacional relevante mediante dashboards.
- Integrar interacción conversacional mediante Telegram.

El foco principal del proyecto está en el diseño del backend, la arquitectura del sistema y el modelado orientado a objetos. No se espera una solución única. Parte importante de la evaluación considera las decisiones de arquitectura y modelado tomadas por cada grupo. **La interfaz web no necesita ser visualmente compleja, pero sí debe ser funcional y usable.**

3.1. Módulos principales

Uno de los objetivos principales del proyecto es modelar correctamente los distintos componentes del dominio utilizando Programación Orientada a Objetos. Se espera que el sistema abstraiga comportamiento común, permita reutilización de lógica y facilite agregar nuevas funcionalidades en el futuro.

El sistema debe soportar inicialmente los siguientes módulos:

3.1.1. Gestión de citas médicas

El sistema debe permitir:

- Crear citas médicas.
- Confirmar citas.
- Cancelar citas.
- Reagendar citas.
- Validar disponibilidad horaria.
- Evitar conflictos de agenda.
- Gestionar estados de citas.

- Manejar horarios de atención.

Cada cita debe poseer un estado claramente definido. Por ejemplo: pendiente, confirmada, cancelada, reagendada, completada, no asistió.

3.1.2. Gestión de agendas

El sistema debe considerar agendas médicas configurables. Debe permitir: definir horarios de atención, bloquear horarios específicos, suspender agendas completas, limitar capacidad de atención, visualizar disponibilidad y validar consistencia de horarios. Las suspensiones de agenda deben afectar correctamente las citas existentes.

3.1.3. Lista de espera

El sistema debe incluir un mecanismo de lista de espera. Debe considerar al menos: inscripción de pacientes en espera, priorización básica, liberación automática de cupos, reasignación de horas disponibles, confirmación de disponibilidad.

3.1.4. Derivaciones

El sistema debe permitir derivar pacientes entre especialidades o profesionales. Para ello, si el médico activó la derivación (por ejemplo, para hacer un examen), se debe enviar un mensaje al paciente y recomendar la derivación. Desde este punto, funciona similar a la toma de horas.

3.1.5. Asistente conversacional IA

El sistema debe integrar un asistente conversacional mediante Telegram. El asistente debe ser capaz de interpretar solicitudes de usuarios, identificar intenciones, extraer información relevante, ejecutar acciones sobre el sistema, responder consultas básicas. Debe poder llevar a cabo las acciones antes descritas: agendar horas, cancelar citas, reagendar, consultar disponibilidad, ingresar a lista de espera. Una forma de realizar esto es crear un prompt que tenga como respuesta una **salida estructurada**.

La lógica principal del sistema NO debe depender exclusivamente del modelo de lenguaje. Las reglas de negocio y validaciones deben implementarse en el backend del sistema.

3.2. Usuarios y permisos

El sistema debe considerar distintos tipos de usuarios:

- **Administradores:** Tienen acceso completo al sistema. Pueden administrar usuarios, especialidades, agendas, configuraciones globales y visualizar toda la información operacional.
- **Recepcionistas:** Pueden gestionar citas, reagendar pacientes, administrar listas de espera y visualizar agendas clínicas.
- **Médicos:** Pueden visualizar su agenda, bloquear horarios, suspender atención y revisar información relacionada con sus pacientes y derivaciones.
- **Pacientes:** Pueden: registrarse, solicitar citas, cancelar horas, reagendar, consultar disponibilidad, interactuar mediante Telegram, ingresar a listas de espera. Un paciente puede tener múltiples citas activas simultáneamente.

3.3. Flujo conversacional

El sistema debe incluir un flujo conversacional automatizado similar al utilizado por asistentes virtuales reales.

Por ejemplo:

1. El paciente solicita una acción mediante Telegram.
2. El sistema interpreta la intención utilizando IA.
3. El backend valida reglas de negocio.
4. El sistema ejecuta la acción correspondiente.
5. El paciente recibe confirmación automática.

El grupo puede proponer otros flujos adicionales según el diseño del sistema.

3.4. Dashboard y visualización

El sistema debe incluir una interfaz web funcional para visualizar información relevante. Se espera como mínimo: visualización de agendas, citas del día, pacientes en espera, cancelaciones, métricas básicas, historial de conversaciones. No se evaluará diseño visual avanzado.

3.5. Testing

El proyecto debe incluir test automatizados. Como mínimo, tests unitarios, tests de lógica de negocio y validación de reglas importantes del sistema. Por ejemplo, validación de conflictos de agenda, cancelaciones inválidas, consistencia de horarios, etc.

3.6. Backend

El backend debe estar desarrollado en Python. Se recomienda el uso de FastAPI, SQLAlchemy y pytest, pero queda a criterio del grupo que stack utilizar, justificando su elección. La aplicación debe incluir persistencia de datos, autenticación básica, integración con Telegram, integración con modelos de lenguaje y despliegue.

3.7. Persistencia de datos

La aplicación debe utilizar una base de datos. No se evaluará la complejidad avanzada del modelo relacional, pero sí que funcione adecuadamente: que sea consistente, con relaciones adecuadas y manejo correcto de datos.

3.8. Interfaz web

La aplicación debe incluir una interfaz web funcional. NO se evaluará diseño visual avanzado, solo que se sea funcional. Se espera como mínimo: navegación básica, formularios funcionales, visualización de información, interacción con el backend.

3.9. Despliegue

La aplicación debe poder ser desplegada fácilmente mediante Docker, sin instalación manual de dependencias. Debe poder mostrarse su funcionamiento, ya sea de forma local o pública. Puede utilizar servicios como: Render, Railway, Vercel u otros. El despliegue debe incluir aplicación funcional, acceso autenticado y persistencia operativa. NO se evaluarán componentes de seguridad informática. Se darán bonificaciones si se despliega en un servicio en la nube.

4. Uso de herramientas de IA

Está permitido el uso de herramientas de inteligencia artificial para apoyar el desarrollo del proyecto. Sin embargo, deben tener en cuenta que el objetivo principal no es únicamente generar código, sino que también diseñar correctamente el sistema, tomar decisiones de arquitectura, modelar adecuadamente el dominio y justificar las soluciones implementadas.

La evaluación y revisión considerará la explicación de sus arquitecturas, justificación de decisiones de diseño, explicación de las relaciones entre clases, descripción de responsabilidades de cada componente y la comprensión completa del código entregado.

5. Entregables

- **Código fuente:** Repositorio en GitHub completo del proyecto. Debe incluir README con instrucciones de ejecución y despliegue, dockerfiles y estructura clara.
- **Prompts utilizados:** Cada uno de los prompts utilizados para la generación del código usando herramientas de IA, indicando a qué parte del código se utilizó.
- **Presentación final:** Deben realizar una presentación final la semana de exámenes, enfocándose en el problema abordado, las decisiones de diseño tomadas y la solución implementada. Se espera que cada grupo muestre el sistema funcionando en vivo, destacando los principales flujos de uso, la arquitectura general, las decisiones técnicas más relevantes y los desafíos enfrentados durante el desarrollo. La evaluación considerará tanto la calidad técnica del sistema como la capacidad del grupo para explicar y defender su solución de manera clara.